

BioSpecInfo

Documentazione dei test — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v180** · commit `0fad440`

Generato il 2026-10-02

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Documentazione dei test — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	bsi-v180
Scopo	Descrivere come sono organizzati i test, come eseguirli, che cosa coprono e dove restano scoperti.

1. Tipologie presenti

Tipo	Presente	Note
End-to-end (E2E)	✓	Playwright su Chromium headless contro un server HTTP locale. È la forma prevalente.
Verifica dei dati	✓	Confronto dei dati scientifici con una fonte indipendente
Prove di stabilità	✓	Sessioni lunghe, memoria esaurita, rete degradata
Coerenza documentazione/codice	✓	Verifica che ciò che la documentazione promette esista davvero
Unit test isolati	✗	Vedi §6
Copertura di codice strumentata	✓	<code>audit_copertura</code> , con il profilatore di Chromium: nessuna build, nessun sorgente riscritto. Vedi §6

Perché E2E e non unit test

L'applicazione è composta da script classici che operano direttamente sul DOM, senza moduli né iniezione delle dipendenze: non esistono unità isolabili senza una riscrittura profonda. I difetti che si sono effettivamente verificati — un canvas sfocato, un pulsante irraggiungibile perché il contenitore è nascosto, un fornitore AI che smette di rispondere — **non sarebbero comunque emersi** da un test unitario: vivono nell'integrazione fra codice, DOM e browser.

La scelta è quindi motivata, non subita. Ne restano i limiti, dichiarati al §6.

2. Come eseguire i test

Requisiti

```
npm install          # solo playwright-core
python3 -m http.server 8899 &  # RDKit WASM richiede contesto HTTP
```

Senza server i test falliscono tutti: aprire i file con `file://` impedisce il caricamento dei moduli WebAssembly.

Tutta la batteria, con rapporto

```
node tools/genera-evidenza.js
```

Esegue ogni banco, ne trascrive l'uscita integrale e produce `docs/evidence/RAPPORTO-VERIFICA.md` con ambiente, commit e impronte SHA-256. Esce con codice `0` solo se nessun banco fallisce.

```
node tools/genera-evidenza.js --veloce # salta i banchi con browser
```

L'opzione `--veloce` è utile per un controllo rapido ma **non è sufficiente per un'attestazione**: i banchi saltati sono proprio quelli che misurano il comportamento reale.

Un singolo banco

```
node tools/verifica-farmaci.js # strumento a sé stante
node tools/verifica-farmaci.js --json # uscita leggibile da programma

cd tools/banchi && node test_spettri.js # un banco E2E
```

I banchi E2E stanno in `tools/banchi/`, versionati come il resto del repository; la variabile `BSI_BANCHI` permette di indicare un'altra cartella.

Non è sempre stato così, ed è bene dirlo. Fino alla versione `bsi-v168` la cartella predefinita era l'area di lavoro della sessione in cui i banchi erano stati scritti: **32 dei 39 banchi non erano nel repository**. Chi clonava e lanciava il comando documentato ne trovava sette, e non riceveva alcun avviso — perché un banco assente veniva contato ma non faceva fallire la batteria, che chiudeva con «CONFORME» e zero fallimenti.

Entrambe le cose sono state corrette: i banchi sono versionati, e **un banco assente rende l'esito NON CONFORME**. Un banco che non c'è non è un banco superato.

3. Composizione della batteria

45 banchi, raggruppati per ciò che dimostrano.

3.1 Dati scientifici

Banco	Verifica	Controlli
verifica-farmaci	Struttura ↔ peso molecolare, duplicati, deviazioni registrate	233 voci
test_spettri	Riconoscimento dei gruppi funzionali su molecole di riferimento	36

Banco	Verifica	Controlli
test_spettri_ui	Innesto del motore nei punti dell'app che disegnano spettri	16
test_assi	Convenzioni degli assi negli spettri SVG	13
test_assi_canvas	Convenzioni degli assi negli spettri su canvas	6
test_costanti	Ricerca delle costanti fisiche e rifiuto delle ambiguità	45
audit_dati	Coerenza dei dati tabulati	29
test_farm_ui	Che l'Atlante Farmaci disegni ogni farmaco che ha in memoria, categoria per categoria, e che una categoria mai etichettata compaia comunque	11
test_simmetria	Gruppi puntuali, modi normali, regole di selezione IR/Raman	26
test_cheminfo	Il motore di chemioinformatica: Tanimoto e Dice su valori calcolabili a mano, impronte di Morgan e MACCS, raggruppamento di Butina, scheletri di Bemis-Murcko, PCA, regressione kernel, divisione per scheletro, modello nullo per rimescolamento delle etichette , validazione incrociata raggruppata, esportazioni CSV e SDF, contrasto dei nove pannelli , accordo fra laboratorio e motore, frammentazione, coppie corrispondenti su effetti noti , ricerca per sottostruttura, tabella SAR, confronto fra modelli nei due versi, arricchimento (EF e BEDROC) su casi calcolabili a mano, intervalli conformi e il loro rifiuto sulla classificazione	189

Un banco che verifica un modello deve verificarlo anche quando il modello deve fallire. test_cheminfo prova il modello nullo nei due versi: su un segnale apprendibile l' R^2 vero (0,394) batte tutti e otto i sosia con etichette rimescolate, con un margine di 0,287; sulle **stesse molecole con etichette casuali** l' R^2 (−0,199) **non** li batte. Un controllo che verifica solo il primo caso passerebbe anche se la guardia fosse cablata su «vero».

Un «contrasto 0» che non copriva la superficie nuova. verifica-accessibilita misura ciò che è visibile, e i sei pannelli della sezione Chemioinformatica restano nascosti finché l'analisi non è stata eseguita: non li aveva mai visti. Ora test_cheminfo esegue l'analisi, apre i pannelli a turno e misura il contrasto su **460 elementi di testo**, con le formule WCAG riscritte dentro il banco — chiedere al codice sotto esame quanto vale il proprio contrasto non è misurare. Il banco pretende inoltre di averne misurati più di 250: se i pannelli non si aprissero, lo zero sarebbe vuoto e il controllo fallirebbe.

3.2 Agente AI

Banco	Condizione simulata
test_404	Il fornitore ha ritirato il modello
test_503	Sovraccarico temporaneo
test_ko · browser_ko	Fornitore irraggiungibile dal browser
test_firma	Firme di ragionamento opache da restituire
test_tetto	Tetto di token appreso dall'errore

Banco	Condizione simulata
test_attesa · test_attesalunga	Timeout e attese imposte
test_doppioinvio	Due invii sovrapposti
browser_prova	Misura di raggiungibilità senza chiave
test_nucleo · browser_prov	Ciclo agentico e selezione del fornitore
caccia_ai	Risposte malformate, stream troncato, 502 con HTML, Stop durante la risposta, pannello chiuso a metà, testo ostile

3.3 Stabilità

Banco	Cosa mette alla prova
audit_stabilita	88 sezioni aperte 5 volte, memoria piena, dati corrotti, raffiche di clic
audit_promesse	Promesse rifiutate e non gestite, con rete sana e con rete morta
audit_quota	localStorage.setItem forzato a fallire su 10 pagine
test_sw	Offline, rete degradata, aggiornamento durante il lavoro
test_filemanager	Archivio non disponibile, spazio esaurito, scritture fallite
test_visore3d	Consumo di contesti WebGL sfogliando molecole

3.4 Interfaccia

browser_reset · browser_proxy · browser_proxyui · browser_rdkit · browser_lab · browser_frontiera · test_aggiorna · test_guidaproxy

3.5 Sicurezza e accessibilità

Banco	Verifica
verifica-sicurezza	Chiavi API nei file tracciati, password in chiaro, segreti nel wrangler.toml , telemetria, script da domini esterni
verifica-accessibilita	Contrasto WCAG AA, nomi accessibili, etichette dei campi, testo alternativo, gerarchia dei titoli, attributo lang — su 13 pagine
audit_mobile	Che a 390 px la pagina non scorra in orizzontale, su tutte le 88 sezioni
audit_rete	Che nessun dato dell'utente lasci il dispositivo : un valore spia seminato in 71 depositi, l'app usata su 6 pagine e 88 sezioni, URL, intestazioni e corpo di ogni richiesta ispezionati
audit_copertura	Quanti byte di JavaScript vengono davvero eseguiti percorrendo l'applicazione: 49,89 %, registrato e difeso

L'ostacolo era dello strumento, non del problema. Per tre versioni la copertura di codice è stata dichiarata non misurabile, con questa motivazione: «strumentare richiederebbe introdurre una build, che l'applicazione non ha». Era vero per `c8` e `istanbul`, che riscrivono il sorgente prima di eseguirlo. Ma **Chromium la raccoglie da solo**, dentro il motore, senza toccare un byte del file servito:

`Profiler.startPreciseCoverage`, che Playwright espone come `page.coverage.startJSCoverage()`. Non serviva una build: serviva accorgersi che il vincolo apparteneva agli strumenti scelti.

Il primo risultato è stato 100 % su ogni file, compresi 3,6 MB di `index.html`. Era il banco che non misurava niente, nella forma più insidiosa: un numero perfetto. V8 emette per ogni funzione un intervallo esterno con il conteggio degli ingressi, e dentro quello gli intervalli a `count: 0` per ciò che non è stato eseguito. Sommando gli intervalli positivi si somma l'intero file. Si fa il contrario: si parte dal totale e si sottrae l'unione dei vuoti.

Il numero vero è **49,89 %**, e regge lo stesso patto del debito di accessibilità: registrato, e il banco fallisce se scende. Una soglia assoluta («80 %») sarebbe una cifra inventata; «non deve peggiorare» è una promessa mantenibile.

Perché una spia e non un elenco di richieste. SEC-02 — «nessun dato personale lascia il dispositivo» — era verificato controllando i due *meccanismi* di uscita: variabile di telemetria vuota, nessuno script da un dominio esterno. È un'evidenza vera ma indiretta: dice «non vedo come potrebbe uscire», che non è «non è uscito». Guardare invece l'elenco degli ospiti contattati sarebbe altrettanto debole, perché conclude per assenza di prove.

La spia rovescia l'onere. Si scrive una stringa irripetibile dentro i dati dell'utente — note, chat, chiavi API, impostazioni, progressi — poi si **usa** l'applicazione, e ogni richiesta che esce viene aperta e letta. Se la stringa non compare da nessuna parte, è perché davvero non è uscita.

```
BSI_PROVA_FUGA=1 node tools/banchi/audit_rete.js # deve FALLIRE
```

Il banco porta con sé la prova di funzionare: con quella variabile provoca una fuga di proposito, verso un ospite **dichiarato** — contattarlo è lecito, mandargli i dati dell'utente no. Se in quel caso non fallisce, è lo strumento a essere guasto.

Perché un banco sul traboccamento orizzontale. UI-03 dice «l'app deve funzionare a 390 px», e la matrice lo dava per verificato da `audit_stabilita`, che però in viewport telefono guarda gli **errori JavaScript**: una pagina può non averne uno solo e uscire lo stesso di centosessantatré pixel dallo schermo. Era un requisito dichiarato coperto e non misurato. Il banco misura `scrollWidth` del documento — esattamente ciò che fa comparire la barra — e non un elemento più largo dello schermo, che dentro un contenitore fatto per scorrere è corretto e riempirebbe l'uscita di rumore finché nessuno la legge più.

Cosa non copre la verifica di accessibilità, e va detto: il testo dentro gli SVG (il colore viene da `fill`, lo sfondo è una forma disegnata) e il testo su sfondi a gradiente (non hanno *un* colore). Gli elementi saltati vengono contati e riportati, non nascosti.

Una sezione per volta. L'ispezione salta gli elementi non visibili — ed è corretto: un elemento nascosto non ha contrasto da misurare. Ma `index.html` alterna 88 sezioni e ne mostra una sola: su **19 751** elementi di testo il banco ne guardava **41**, e stampava «0 difetti». Non era un risultato falso, era un risultato su un campione che nessuno aveva dichiarato. Ora le sezioni vengono aperte una per una, e **il numero di sezioni percorse viene stampato**.

Quel conteggio si è guadagnato lo stipendio alla prima esecuzione: il primo tentativo ne percorreva **zero** — il clic di Playwright aspetta che l'elemento sia visibile, e i pulsanti stanno dentro gruppi di navigazione richiusi — e il banco l'ha detto («88 sezioni presenti, nessuna percorsa») invece di stampare un altro zero rassicurante.

3.5-bis Il debito di accessibilità, e il patto che non cresca

Percorrendo tutte le sezioni sono emersi **1 069** difetti di contrasto. Oggi sono **zero** — l'elenco delle cause e dei rimedi è in `docs/09 §4` — e i campi privi di etichetta sono scesi da 40 a **zero**.

Il meccanismo descritto qui sotto resta, e conta più del numero che custodisce. Pretendere zero *da subito* avrebbe lasciato due sole vie, entrambe cattive: la verifica rossa per sempre, oppure allentata finché torna verde. La terza via è **dichiarare il numero** e impedirgli di crescere, qualunque esso sia. È così che da 1 069 si è arrivati a zero: ogni scalino registrato, nessuno annullabile di nascosto.

File	Ruolo
<code>docs/evidence/accessibilita-riferimento.json</code>	Il conteggio registrato

Situazione	Esito del banco
Il conteggio sale	✗ FALLITO — è una regressione
Il conteggio scende	✓ passa, e chiede di aggiornare il riferimento
Il conteggio è uguale	✓ passa in silenzio: debito dichiarato, non cresciuto — oggi quel valore è 0 , e il banco lo difende

```
node tools/verifica-accessibilita.js --aggiorna-riferimento
```

Da usare **solo dopo aver ridotto** i difetti, mai per far tornare verde una regressione. Anche a zero non è conformità WCAG piena: restano fuori il testo dentro gli SVG e quello su fondo a gradiente, che l'automatismo non sa giudicare e che il banco **conta e riporta** a ogni esecuzione (`docs/09 D-09`). È la misura onesta di ciò che si può misurare, con la garanzia che non peggiori di nascosto.

3.6 Coerenza

Banco	Verifica
verifica_guida	Che le promesse della documentazione esistano nel codice; assenza di marcatori di conflitto su 58 file
verifica-documenti	Collegamenti interni, allineamento delle versioni, coerenza dell'indice, motivazione delle deviazioni, coerenza interna della matrice di tracciabilità (identificativi non duplicati, totali di copertura pari agli identificativi realmente definiti)
genera-pacchetti	Riscrive i riferimenti dei documenti estratti e verifica che nessun collegamento resti rotto nei tre pacchetti di consegna
genera-pdf	Converte i documenti in PDF e fallisce se un file risulta sotto la soglia di plausibilità (conversione a vuoto)
verifica-affermazioni	Confronta ogni numero dichiarato nei documenti — sezioni, farmaci, malattie, tumori, strategie, moduli — con quello misurato nell'applicazione in esecuzione , in italiano e in inglese

Perché i numeri vanno misurati, non ricordati. Il dossier tecnico si apre con delle cifre: «63 malattie (23 tumori)», «46 strategie», «25 moduli». Sono la prima cosa che legge chi valuta, e bastano perché tutto il resto venga creduto o messo in dubbio. Nessuno le controllava: scritte una volta, restavano, mentre l'applicazione cambiava. E infatti chimorga.html dichiarava «**25 moduli**» in cima e «**17 moduli**» più sotto — due affermazioni sullo stesso oggetto, nello stesso file, una delle due falsa.

Una lezione pagata scrivendo il banco. Avevo già concluso che «63 malattie» fosse un'esagerazione: nel sorgente avevo trovato un array DISEASES con 12 voci, e stavo per «correggere» il documento. Misurando nel DOM ho visto che il menu ne elenca davvero 63 — l'array che avevo trovato era un altro, più piccolo. **Un conteggio sul sorgente non è una misura: è un indizio.** Per questo il banco apre la pagina, apre la sezione e conta. Controlla anche che i documenti concordino **fra loro**: due documenti che dicono cose diverse sullo stesso oggetto sono un difetto anche quando uno dei due ha ragione. La traduzione inglese conta come uno di quei documenti — docs/en/05 è rimasto a «84 sections» e fermo alla versione bsi-v146 per tre rilasci, e nessun banco se ne accorgeva perché nessuno guardava l'insieme inglese.

Perché un controllo sulla matrice. La matrice dichiarava SCI-10 due volte — una come requisito verificato al 100 %, una come requisito esplicitamente **non** verificato — e la tabella riassuntiva sommava 36 requisiti mentre nel documento ne erano definiti 37. Entrambi gli errori stavano nel numero che un valutatore legge per primo. Ora il banco conta gli identificativi realmente presenti e fallisce se i totali scritti non corrispondono: la prova che il controllo misura davvero è che, reintrodotti i due errori uno per volta, li segnala entrambi.

4. Casi di prova notevoli

Alcuni banchi meritano una menzione perché verificano proprietà che un test funzionale non osserva.

Caso	Metodo	Risultato misurato
Perdite di memoria	88 sezioni × 5 giri, nodi DOM contati a ogni giro	+26 876 al primo giro (costruzione), +0 nei quattro successivi
Memoria esaurita	<code>setItem</code> sostituito con una funzione che lancia sempre	10 pagine su 10 restano operative
Rete degradata ≠ assente	Richieste sospese 20 s, intercettazione a livello di contesto	Risposta dalla cache in 3 507 ms (soglia 3 500)
Riproducibilità degli spettri	Doppio disegno, confronto byte a byte	Identici
Contesti WebGL	Costruzioni del visore su 10 molecole consecutive	Da 7 a 1
Cronologia corrotta	Due <code>Invio</code> a 500 ms di distanza	<code>user,assistant</code> anziché <code>user,user,assistant,assistant</code>
Contrasto del testo	Formula WCAG su ogni elemento con testo proprio, 13 pagine e 88 sezioni	19 751 elementi esaminati (erano 41): 1 069 difetti emersi, 1 069 corretti , 0 registrati come valore che non può crescere
Annullamento immediato	Stop premuto a 1,5 s, stato campionato ogni secondo	pulsante <code>Invia</code> disponibile dal 1° secondo (era il 10°)

5. Come aggiungere un banco

Struttura minima

```
/* Un commento che spiega COSA dimostra questo banco e perché serve. */
const { chromium } = require('playwright-core');
const CHROME = '/opt/pw-browsers/chromium-1194/chrome-linux/chrome';

(async () => {
  const b = await chromium.launch({ executablePath: CHROME, args: ['--no-sandbox'] });
  // serviceWorkers: 'block' se il banco modifica i file: senza, il SW
  // potrebbe servire una copia in cache e si misurerebbe il file vecchio
  const ctx = await b.newContext({ serviceWorkers: 'block' });
  const pg = await ctx.newPage();
  const err = [];
  pg.on('pageerror', e => err.push(e.message));

  let ok = 0, ko = 0;
  const att = (d, atteso, avuto) => {
    if (String(atteso) === String(avuto)) { ok++; console.log(' ✓ ' + d + ' → ' + avuto); }
    else { ko++; console.log(' x ' + d + '\n      atteso: ' + atteso + '\n      avuto: ' +
avuto); }
  };

  await pg.goto('http://127.0.0.1:8899/index.html', { waitUntil: 'load', timeout: 60000 });
  await pg.waitForTimeout(2500);

  // ... misure ...

  att('nessun errore JS', 0, err.length);
  await b.close();
  console.log('\n' + (ko ? 'x ' + ko + ' FALLITI, ' : '') + ok + ' passati');
  process.exit(ko ? 1 : 0); // il codice di uscita è ciò che conta
})();
```

Poi si aggiunge il nome alla famiglia opportuna in `tools/genera-evidenza.js`.

La regola che conta

Un banco che non misura nulla passa. È il modo più insidioso di ottenere una falsa garanzia: nessun allarme, nessun fallimento, e nessuna verifica.

Perciò **ogni banco che simula una condizione deve contare quante volte la simulazione è realmente scattata, e fallire se il conteggio è zero.**

Casi realmente incontrati durante lo sviluppo della batteria:

Trappola	Come si manifestava
<code>page.route</code> non intercetta le richieste del Service Worker (serve <code>context.route</code>)	La prova «rete degradata» misurava una rete sana e concludeva in 16 ms invece di 3 500
Sondare <code>localStorage</code> esaurito con una scrittura di un byte	Dopo la saturazione un byte trova sempre posto: il banco «passava»

Trappola	Come si manifestava
Cercare <code>[data-tab]</code> dove la navigazione usa <code>[data-s]</code>	Zero schede trovate, quindi zero misure — e zero fallimenti
Le tacche di un asse su <code><canvas></code> non sono testo	Vanno raccolte intercettando <code>fillText</code> ; e la riga più in basso è il <i>titolo</i> dell'asse, non le tacche
Sonda installata su una libreria caricata dopo	Viene sostituita insieme alla libreria: zero chiamate contate mentre il codice funzionava
Una <code>var</code> dentro una IIFE non è <code>window.qualcosa</code>	Assegnarla da fuori crea una variabile diversa

6. Lacune di copertura

Dichiarate. La copertura riportata in `08-Traceability-Matrix.md` misura **quanti requisiti hanno un banco**, non quante righe di codice vengono eseguite: sono due cose diverse e non vanno confuse.

Lacuna	Situazione attuale	Raccomandazione
Copertura di codice	Misurata: 49,89 % di istruzioni sul percorso più ampio. Resta fuori la copertura dell'intera batteria e quella di rami : un <code>if</code> entrato da un solo lato conta come coperto	Estendere la raccolta a ogni banco, e passare dalla copertura di istruzioni a quella di rami
Accessibilità	Automatizzata su 13 pagine e tutte le 88 sezioni: contrasto WCAG, nomi accessibili, etichette, testo alternativo, gerarchia dei titoli. Restano fuori il testo negli SVG e quello su gradiente, contati a ogni esecuzione	Affiancare <code>axe-core</code> per le regole che questo banco non implementa (ruoli ARIA, ordine di tabulazione, gestione del fuoco)
Sicurezza	<code>verifica-sicurezza</code> esegue 9 controlli su 235 file tracciati e copre SEC-01, SEC-03, SEC-05, SEC-06; SEC-04 è coperto da <code>verifica_guida</code> . SEC-02 resta indiretto : vedi <code>docs/09 D-03</code>	Osservare il traffico di rete durante un uso reale, l'unica verifica diretta di SEC-02
Browser diversi da Chromium	Nessuna prova automatica su Firefox o WebKit	Estendere i banchi principali a <code>webkit</code> , dove le differenze su IndexedDB e Service Worker sono maggiori
Prestazioni	Prove manuali cross-device	Misura automatica del tempo di primo disegno
Regressione visiva	Assente	Confronto di schermate per i grafici, che sono il cuore del prodotto

7. Criteri di accettazione

Una versione non viene pubblicata se uno solo di questi non è soddisfatto.

- ☐ `node tools/genera-evidenza.js` si chiude con **0 falliti**

- ☐ `tools/verifica-farmaci.js` : nessun difetto, ogni deviazione registrata con il motivo
 - ☐ `verifica_guida` : nessuna promessa della documentazione priva di riscontro
 - ☐ Nessun errore JavaScript non di rete su tutte le pagine
 - ☐ Rapporto di evidenza rigenerato sul commit che si pubblica
-

Documento aggiornato alla versione `bsi-v180`.